



Мои финансы

Персональный финансовый дашборд студента

Техническое задание

1. Описание задачи

1.1. Проблема

Знакомая ситуация: до стипендии ещё неделя, а деньги уже закончились. Куда они ушли — непонятно. Кажется, ничего особенного не покупал, но баланс на карте говорит обратное.

Управление личными финансами — одна из главных сложностей студенческой жизни. Большинство студентов не ведёт учёт своих расходов и доходов, а если и начинает — бросает через пару дней, потому что существующие инструменты слишком сложные или не заточены под студенческую реальность.

1.2. Что нужно создать

Веб-приложение «Мои финансы» — простой и удобный инструмент, который помогает студенту:

- Понять, куда уходят деньги
- Вести учёт доходов и расходов
- Видеть общую картину своих финансов

Приложение должно быть настолько простым, чтобы им хотелось пользоваться каждый день.

2. Обязательный функционал

Для успешной сдачи проекта необходимо реализовать **все три фичи** ниже.

2.1. Фича 1: Добавление транзакций

Пользователь может добавить доход или расход. Для каждой транзакции указывается:

- Тип: доход или расход
- Сумма (число)
- Категория (еда, транспорт, развлечения, учёба, другое)
- Дата
- Короткий комментарий (необязательно)

💡 Подсказка:

Создайте HTML-форму с полями ввода. Для хранения данных достаточно использовать `localStorage` браузера — данные сохраняются даже после перезагрузки страницы.

Пример сохранения массива в `localStorage`:

```
// Сохранить
```

```
localStorage.setItem('transactions', JSON.stringify(transactions));

// Прочитать
const transactions = JSON.parse(
  localStorage.getItem('transactions')
) || [];
```

 [MDN — Использование localStorage](#)

2.2. Фича 2: Список транзакций

Отображение всех добавленных транзакций в виде таблицы или списка. Каждая запись показывает:

- Дату
- Категорию
- Сумму (с визуальным отличием для доходов и расходов)
- Комментарий
- Кнопку удаления

Подсказка:

Для отображения списка динамически создавайте HTML-элементы через JavaScript. Доходы и расходы визуально разделите цветом (зелёный для доходов, красный для расходов).

Простой способ — шаблонные строки:

```
const html = transactions.map(t => `
  <div class="${t.type === 'income' ? 'green' : 'red'}">
    ${t.date} | ${t.category} | ${t.amount} p.
  </div>
`).join('');
document.getElementById('list').innerHTML = html;
```

 [MDN — Манипуляции с DOM](#)

2.3. Фича 3: Общая статистика

В верхней части страницы отображаются три ключевых показателя:

- **Текущий баланс** — сумма всех доходов минус сумма всех расходов
- **Доходы за месяц** — сумма доходов за текущий месяц
- **Расходы за месяц** — сумма расходов за текущий месяц

Подсказка:

Посчитайте суммы с помощью метода `Array.reduce()` по массиву транзакций.

Выведите три числа крупным шрифтом — это и будет ваш «дашборд».

```
const totalExpenses = transactions
  .filter(t => t.type === 'expense')
  .reduce((sum, t) => sum + t.amount, 0);
```

 [MDN — Array.prototype.reduce\(\)](#)

3. Бонусный функционал

★ Бонусный функционал (за дополнительные баллы):

Эти фишки не обязательны, но принесут дополнительные баллы при оценке:

- **Фильтрация транзакций** — по категории, дате или типу (доходы/расходы)
- **Экспорт в CSV** — кнопка для скачивания всех транзакций в виде файла
- **Тёмная тема** — переключатель светлой и тёмной темы
- **Редактирование транзакции** — возможность изменить уже добавленную запись
- **Сортировка** — по дате, сумме или категории

4. Критерии оценки

4.1. Таблица критериев

Критерий	Описание	Баллы
Функциональность	Все 3 обязательные фишки работают корректно	30
UX (удобство)	Приложением удобно пользоваться, интерфейс понятен без инструкции	15
UI (дизайн)	Аккуратный, визуально приятный интерфейс	15
Качество кода	Читаемый код, понятные имена переменных, наличие комментариев	10
Бонусный функционал	Реализованы дополнительные фишки	20
Защита	Демо, объяснение решений, ответы на вопросы жюри	10
Итого		100

4.2. Пояснения

- **Функциональность:** каждая из 3 фишек оценивается отдельно. Частичная реализация — частичные баллы.
- **UX:** можно ли сразу понять, как добавить транзакцию? Видно ли баланс? Не путает ли интерфейс?

- **UI:** не требуется wow-дизайн. Достаточно аккуратности: ровные отступы, читаемые шрифты, приятные цвета.
- **Код:** комментарии приветствуются. Один огромный файл — хуже, чем разделение на логические блоки.
- **Защита:** покажите живую работу приложения, расскажите что было сложнее всего, кто за что отвечал.

4.3. Что оценивается на защите

1. Демонстрация работающего приложения (живой показ, не скриншоты)
2. Объяснение: какую проблему решили и почему выбрали такой подход
3. Ответы на вопросы жюри (технические и продуктовые)
4. Командная работа: кто за что отвечал в команде

5. Рекомендуемый стек технологий

⚠ Важно:

Вы можете использовать **любые** технологии на ваш выбор. Ниже — рекомендации, которые помогут быстрее начать и не тратить время на выбор инструментов.

- **Языки:** HTML + CSS + JavaScript (без фреймворков)
- **Хранение данных:** localStorage (встроено в браузер, ничего устанавливать не нужно)
- **Редактор кода:** VS Code ([скачать бесплатно](#))
- **Запуск проекта:** откройте index.html в браузере или используйте расширение Live Server в VS Code
- **Структура проекта:**

```
my-finances/  
  index.html      <- главная страница  
  style.css       <- стили  
  script.js       <- логика
```

6. Полезные ресурсы

📁 Основы веб-разработки

- [MDN Web Docs](#) — справочник по HTML, CSS, JavaScript (на русском)
- [FreeCodeCamp](#) — бесплатные интерактивные tutorиалы
- [HTML Academy](#) — курсы по вёрстке на русском

🗄️ Работа с данными в браузере

- [MDN — localStorage](#)
- [Learn JavaScript — localStorage \(на русском\)](#)

🖌️ CSS и внешний вид

- [Bootstrap 5](#) — подключите через CDN для быстрого оформления
- [Google Fonts](#) — бесплатные шрифты
- [Coolors](#) — генератор цветовых палитр

💡 Дизайн и вдохновение

- [Dribbble](#) — «finance dashboard»
- [Behance](#) — «budget app»

📺 Видеоуроки

- [YouTube](#) — «expense tracker html css javascript»

7. Подсказки по порядку разработки

⚠️ Важно:

Этот план — рекомендация, а не обязательство. Работайте так, как удобно вашей команде. Но если не знаете с чего начать — следуйте этим шагам.

7.1. Этап 1: Каркас

1. Создайте три файла: `index.html`, `style.css`, `script.js`
2. Сверстайте главный экран:
 - Шапка с названием приложения
 - Три карточки сверху (баланс, доходы, расходы) — пока с нулями

- Форма добавления транзакции
- Пустой блок для списка транзакций

3. Не гонитесь за красотой — сначала пусть всё отображается

7.2. Этап 2: Логика добавления

4. Обработайте отправку формы: при нажатии кнопки «Добавить» — соберите данные из полей
5. Создайте объект транзакции и добавьте его в массив
6. Сохраните массив в `localStorage`
7. Отобразите новую транзакцию в списке на странице

7.3. Этап 3: Статистика и удаление

8. Реализуйте подсчёт баланса, доходов и расходов за месяц
9. Обновляйте три карточки при каждом изменении данных
10. Добавьте кнопку удаления для каждой транзакции
11. При загрузке страницы — читайте данные из `localStorage` и отображайте их

7.4. Этап 4: Оформление

12. Приведите дизайн в порядок: отступы, цвета, шрифты
13. Сделайте доходы зелёными, расходы красными
14. Добавьте бонусные фичи, если есть время
15. Протестируйте: добавление, удаление, перезагрузка страницы

7.5. Этап 5: Подготовка к защите

16. Подготовьте демо-сценарий: что будете показывать жюри
17. Добавьте несколько тестовых транзакций для наглядности
18. Залейте код на GitHub (необязательно, но приветствуется)
19. Распределите, кто что рассказывает на защите

8. Если застряли

- **Не работает localStorage:**

Проверьте, что вы сериализуете данные через `JSON.stringify()` при сохранении и `JSON.parse()` при чтении. Без этого массив превратится в строку "[object Object]".

- **Данные пропадают при перезагрузке:**

Убедитесь, что при загрузке страницы вы читаете данные из `localStorage` и отображаете их. Добавьте в начало `script.js`:

```
window.addEventListener('load', () => {
  const saved = localStorage.getItem('transactions');
  if (saved) {
    transactions = JSON.parse(saved);
    renderTransactions();
    updateStats();
  }
});
```

- **Форма перезагружает страницу при отправке:**

Добавьте `event.preventDefault()` в обработчик формы:

```
form.addEventListener('submit', (e) => {
  e.preventDefault();
  // ваш код добавления транзакции
});
```

- **Не знаете как начать:**

Начните с самого простого: одна HTML-страница, одна кнопка, один `alert('Работает!')`. Убедитесь, что JavaScript подключён, и усложняйте постепенно.

- **Проблемы с дизайном:**

Подключите Bootstrap через CDN (одна строка `<head>`) — получите аккуратные кнопки, формы и таблицы без усилий.

[Bootstrap — Quick Start](#)

9. Формат сдачи

9.1. Что нужно предоставить

1. **Работающий прототип** — приложение, которое можно открыть и продемонстрировать с ноутбука
2. **Исходный код** — папка с файлами проекта или ссылка на GitHub-репозиторий

9.2. Как проходит защита

1. Команда показывает работающее приложение жюри (живой показ, не скриншоты)
2. Проходит основной сценарий: добавление транзакции → просмотр списка → просмотр статистики
3. Рассказывает, какие решения принимали и почему
4. Отвечает на вопросы жюри

Удачи! 🚀
Команда Т-Банк